Relazione Tecnica sullo Sviluppo di una Binary Heap Generica in C

22 marzo 2026

Indice

1 Introduzione **3**

2 Analisi Preliminare dei Sorgenti glibc **3**
2.1 Accesso ai sorgenti . . . 3
2.2 Ricerca di una binary heap o priority queue già presente . . . 3
2.3 Esito della ricerca . . . 4
2.4 La coda ordinata dei timer . . . 4
2.5 Il riferimento utile: `frame_heapsort` . . . 4

3 Scelte Progettuali **5**
3.1 Obiettivo della nuova implementazione . . . 5
3.2 Rappresentazione dei dati . . . 5
3.3 Convenzione del comparatore . . . 5
3.4 Perchè una min-heap . . . 6

4 Implementazione del File `binaryheap.c` **6**
4.1 Interfaccia pubblica . . . 6
4.2 `binaryheap_init` . . . 6
4.3 `binaryheap_destroy` . . . 7
4.4 `binaryheap_reserve` . . . 7
4.5 `binaryheap_push` . . . 7
4.6 `binaryheap_peek` . . . 7
4.7 `binaryheap_pop` . . . 7
4.8 `binaryheap_size` e `binaryheap_empty` . . . 8
4.9 `binaryheap_build` . . . 8

5 Dettagli Algoritmici **8**
5.1 `binaryheap_sift_up` . . . 8
5.2 `binaryheap_sift_down` . . . 8
5.3 Invariante della struttura . . . 9

6 Complessità Computazionale **9**

7 Programma Demo **9**
7.1 Scopo di `binaryheap_demo.c` 9
7.2 Differenza tra demo e test 10

8 Testing **10**
8.1 Motivazione 10
8.2 File `test_binaryheap.c` 10
8.3 Output dei test 10

9 Build Automation con Makefile **11**
9.1 Obiettivi del Makefile 11
9.2 Target disponibili 11
9.3 Scelta delle flag di compilazione 11

10 Struttura Finale dei File **11**

11 Valutazione del Lavoro Svolto **12**
11.1 Risultati raggiunti 12
11.2 Limiti attuali 12
11.3 Possibili estensioni 12

12 Conclusione **12**

1 Introduzione

L'obiettivo del lavoro svolto in questa directory è stato progettare e implementare una struttura dati di tipo *binary heap* generica in linguaggio C, prendendo come punto di partenza un'analisi dei sorgenti della `glibc` 2.36 presenti nel progetto tramite symlink. Il risultato finale comprende:

- un'implementazione generica della heap basata su `void **` e funzione di confronto;
- un file header con l'interfaccia pubblica;
- un programma dimostrativo eseguibile;
- un file di test separato con controlli automatici;
- un `Makefile` per build, esecuzione della demo e lancio dei test.

La richiesta iniziale non era soltanto "scrivere una heap", ma capire prima se all'interno della `glibc` fosse già presente una struttura dati simile o riusabile. Per questo motivo il lavoro è partito da una fase di esplorazione del codice sorgente, seguita poi da una fase di progettazione e infine dalla realizzazione vera e propria.

2 Analisi Preliminare dei Sorgenti glibc

2.1 Accesso ai sorgenti

All'interno della directory di lavoro era presente un symlink chiamato `glibc-2.36`, puntante ai sorgenti reali della libreria GNU C. La prima attività è consistita nel verificare:

- che il symlink fosse leggibile;
- che il target fosse effettivamente accessibile dal workspace;
- che la directory contenesse i file tipici dei sorgenti `glibc`, come `configure`, `Makerules`, `README` e `COPYING`.

La verifica ha confermato che il sorgente era disponibile e ispezionabile.

2.2 Ricerca di una binary heap o priority queue già presente

La seconda fase è stata una ricerca mirata nei sorgenti `glibc` per stabilire se esistesse già:

- una struttura dati dichiarata esplicitamente come *binary heap*;
- una *priority queue* riusabile;
- una utility generica che implementasse operazioni equivalenti;
- un'implementazione locale che potesse essere estratta o adattata.

La ricerca è stata condotta usando parole chiave come:

- `binary heap`;
- `priority queue`, `priority_queue`, `pqueue`, `pq`;
- `heapify`, `sift_down`, `sift_up`;
- nomi file contenenti `heap`, `priority`, `queue`.

2.3 Esito della ricerca

La conclusione dell'analisi è stata netta:

- nei sorgenti `glibc-2.36` non è presente una *priority queue* generica pronta all'uso;
- non è presente una struttura dati esposta o documentata come *binary heap* riusabile;
- esistono però due elementi interessanti:
  - una coda ordinata per timer, implementata come lista ordinata;
  - un'implementazione locale di *heapsort* che utilizza esplicitamente la proprietà di heap binario su array.

2.4 La coda ordinata dei timer

Nel file `fbtl/sysdeps/pthread/timer_routines.c` è presente una struttura che, a prima vista, ricorda una priority queue: i timer vengono inseriti in ordine di scadenza. Tuttavia l'implementazione non usa un heap, ma una lista doppiamente collegata ordinata. L'inserimento avviene tramite scansione lineare fino al punto corretto.

Questo approccio ha caratteristiche diverse rispetto a una binary heap:

- inserimento in tempo lineare $O(n)$;
- accesso al minimo in tempo costante $O(1)$, essendo in testa;
- nessuna rappresentazione su array;
- nessun *sift up* o *sift down*.

Per questo motivo tale codice non era un punto di partenza adeguato per implementare una heap binaria vera e propria.

2.5 Il riferimento utile: `frame_heapsort`

Nel file `sysdeps/generic/unwind-dw2-fde.c` è stata individuata la funzione `frame_heapsort`, che usa chiaramente la rappresentazione classica di un heap binario su array:

- figlio sinistro all'indice $2i + 1$;
- figlio destro all'indice $2i + 2$;
- ripristino della proprietà di heap tramite scambi successivi;
- selezione del figlio con priorità maggiore per mantenere l'invariante.

Pur essendo specializzata per il contesto interno della `glibc` e per una funzione di ordinamento, questa implementazione conteneva l'idea algoritmica giusta per derivare una struttura dati generica.

3 Scelte Progettuali

3.1 Obiettivo della nuova implementazione

L'obiettivo è stato definire una heap riusabile in C con queste proprietà:

- genericità sugli elementi memorizzati;
- separazione tra struttura dati e criterio di ordinamento;
- API semplice e minimale;
- dipendenze standard minime;
- integrazione facile con il `Makefile`.

3.2 Rappresentazione dei dati

La heap è stata implementata come array dinamico di puntatori generici:

```
struct binaryheap
{
    void **data;
    size_t size;
    size_t capacity;
    binaryheap_cmp_fn cmp;
};
```

Questa scelta comporta diversi vantaggi:

- la struttura non copia gli oggetti, ma memorizza puntatori;
- può contenere elementi di qualsiasi tipo;
- il criterio di priorità è delegato a una funzione di confronto;
- l'allocazione e la crescita della memoria sono gestite in modo centralizzato.

3.3 Convenzione del comparatore

Il comparatore segue una convenzione in stile `qsort`:

- ritorno < 0: il primo argomento ha priorità più alta;
- ritorno $= 0$: i due elementi sono equivalenti;
- ritorno > 0: il secondo argomento ha priorità più alta.

Con questa convenzione, l'implementazione realizzata si comporta come una *min-heap*: l'elemento "più piccolò' secondo il comparatore risale verso la radice.

3.4 Perchè una min-heap

La scelta di implementare una min-heap e non una max-heap deriva da motivi pragmatici:

- molti usi tipici della priority queue richiedono l'estrazione del minimo;
- il comportamento è naturale se il comparatore ordina in senso crescente;
- passare da min-heap a max-heap è comunque banale: basta invertire la logica del comparatore.

4 Implementazione del File `binaryheap.c`

4.1 Interfaccia pubblica

Per rendere l'implementazione usabile sia dal programma demo sia dal file di test, è stato creato anche il file `binaryheap.h`, che espone:

- il tipo funzione `binaryheap_cmp_fn`;
- la struttura `struct binaryheap`;
- le funzioni pubbliche della heap.

Le operazioni esposte sono:

- `binaryheap_init`;
- `binaryheap_destroy`;
- `binaryheap_reserve`;
- `binaryheap_push`;
- `binaryheap_peek`;
- `binaryheap_pop`;
- `binaryheap_size`;
- `binaryheap_empty`;
- `binaryheap_build`.

4.2 `binaryheap_init`

Questa funzione inizializza la struttura in uno stato vuoto:

- puntatore dati a `NULL`;
- dimensione a zero;
- capacità a zero;
- salvataggio del comparatore.

L'inizializzazione non alloca memoria subito. La memoria viene richiesta solo al primo inserimento o in seguito a una `reserve`.

4.3 `binaryheap_destroy`

Questa funzione libera la memoria allocata dinamicamente per l'array interno. Non dealloca gli elementi puntati, perchè la heap non ne è proprietaria. Questa è una scelta intenzionale: la struttura gestisce i puntatori, non il ciclo di vita degli oggetti referenziati.

4.4 `binaryheap_reserve`

Questa funzione permette di preallocare memoria per almeno un certo numero di elementi. Se la capacità richiesta è inferiore o uguale a quella già disponibile, non viene eseguita alcuna riallocazione.

Se invece serve aumentare la memoria:

- viene invocata `realloc`;
- in caso di successo si aggiorna la capacità;
- in caso di fallimento si ritorna `-1`.

La funzione è utile sia per ottimizzazione, sia come building block per `push`.

4.5 `binaryheap_push`

L'inserimento di un elemento segue due passi:

1. se necessario, espande la capacità dell'array;
2. inserisce il nuovo elemento in fondo e lo fa risalire tramite *sift up*.

La crescita della capacità segue una politica geometrica:

- da 0 a 8 elementi alla prima espansione;
- successivamente raddoppio della capacità.

Questa scelta riduce il numero di riallocazioni e fornisce costo ammortizzato efficiente sugli inserimenti.

4.6 `binaryheap_peek`

Restituisce l'elemento in radice senza rimuoverlo. Se la heap è vuota, ritorna `NULL`. In una min-heap questo elemento corrisponde al minimo secondo il comparatore.

4.7 `binaryheap_pop`

L'estrazione dell'elemento in testa avviene come segue:

1. salva l'elemento in radice;
2. sposta l'ultimo elemento della heap in posizione 0;
3. decrementa la dimensione;
4. ripristina la proprietà di heap con *sift down*.

Se la heap è vuota, ritorna `NULL`.

4.8 `binaryheap_size` e `binaryheap_empty`

Queste funzioni forniscono introspezione minima sulla struttura:

- `binaryheap_size` restituisce il numero di elementi;
- `binaryheap_empty` ritorna vero se la heap è vuota.

4.9 `binaryheap_build`

Questa funzione costruisce una heap da un array di puntatori già esistente. Invece di inserire gli elementi uno a uno, applica una costruzione bottom-up:

- inizializza i campi della struttura;
- parte dall'ultimo nodo interno;
- applica *sift down* fino alla radice.

Questa tecnica permette una costruzione in tempo lineare $O(n)$, più efficiente rispetto a n inserimenti consecutivi.

5 Dettagli Algoritmici

5.1 `binaryheap_sift_up`

L'operazione di risalita confronta un nodo con il padre:

- se il nodo ha priorità più alta del padre, avviene uno scambio;
- il processo continua finchè l'invariante di heap non è ristabilito o si raggiunge la radice.

Nel caso peggiore, il numero di passi è proporzionale all'altezza dell'albero, quindi $O(\log n)$.

5.2 `binaryheap_sift_down`

L'operazione di discesa confronta un nodo con i figli:

- viene scelto il figlio con priorità maggiore tra i due figli disponibili;
- se tale figlio ha priorità maggiore del nodo corrente, i due vengono scambiati;
- l'algoritmo continua dal nuovo indice.

Anche in questo caso il costo nel caso peggiore è $O(\log n)$.

5.3 Invariante della struttura

L'invariante fondamentale della heap è:

$$\text{per ogni indice } i, \quad data[i] \leq data[2i+1] \text{ e } data[i] \leq data[2i+2]$$

naturalmente nel senso definito dal comparatore.

Questo garantisce che la radice contenga sempre l'elemento con priorità più alta nel senso scelto dalla struttura, cioè il minimo nel caso corrente.

6 Complessità Computazionale

Le complessità delle operazioni principali sono:

Operazione	Complessità
`binaryheap_init`	$O(1)$
`binaryheap_destroy`	$O(1)$
`binaryheap_peek`	$O(1)$
`binaryheap_empty`	$O(1)$
`binaryheap_size`	$O(1)$
`binaryheap_push`	$O(\log n)$ ammortizzato
`binaryheap_pop`	$O(\log n)$
`binaryheap_build`	$O(n)$

L'uso di un array dinamico rende inoltre l'accesso ai nodi molto efficiente dal punto di vista cache, rispetto a strutture basate su puntatori espliciti a figli e genitori.

7 Programma Demo

7.1 Scopo di `binaryheap_demo.c`

È stato creato un programma dimostrativo, `binaryheap_demo.c`, per mostrare l'uso base della struttura:

- definizione di un comparatore per interi;
- inizializzazione della heap;
- inserimento di alcuni valori;
- estrazione in ordine crescente;
- stampa dei risultati.

Questo file non nasce come test formale, ma come esempio d'uso e verifica manuale rapida.

7.2 Differenza tra demo e test

La demo e il test hanno scopi diversi:

- la demo mostra *come usare* l'API;
- il test controlla *se l'API funziona correttamente*.

Per questo motivo la demo è stata lasciata separata dal file di test.

8 Testing

8.1 Motivazione

Dopo la demo, è emersa la necessità di avere un file di test separato. Il motivo principale è che una demo può dare l'impressione che il codice funzioni, ma non fornisce garanzie sistematiche. Un test dedicato invece:

- verifica più scenari;
- fallisce automaticamente in presenza di errori;
- è integrabile in un workflow di build;
- può essere esteso facilmente.

8.2 File `test_binaryheap.c`

È stato quindi aggiunto il file `test_binaryheap.c`, costruito con `assert`. I casi coperti sono:

- **heap vuota**: verifica di `empty`, `size`, `peek` e `pop` su struttura inizializzata ma senza elementi;
- **ordine di inserimento/estrazione**: inserimento di una sequenza non ordinata e controllo dell'estrazione in ordine crescente;
- **duplicati**: verifica del corretto comportamento in presenza di valori ripetuti;
- **build**: costruzione di una heap a partire da un array già esistente e verifica dell'ordine finale.

8.3 Output dei test

Inizialmente il file di test non produceva output in caso di successo, comportamento corretto ma poco evidente a schermo. Successivamente è stato aggiunto un messaggio finale:

```
All tests passed
```

In questo modo:

- se un `assert` fallisce, il programma termina con errore;
- se tutto va bene, il test stampa esplicitamente il successo.

9 Build Automation con Makefile

9.1 Obiettivi del Makefile

Il `Makefile` è stato creato per gestire in modo semplice e ripetibile le attività essenziali:

- compilazione del file oggetto;
- compilazione ed esecuzione della demo;
- compilazione ed esecuzione dei test;
- pulizia dei file generati.

9.2 Target disponibili

I target principali sono:

- `make`: compila `binaryheap.o`;
- `make run`: compila ed esegue `binaryheap_demo`;
- `make test`: compila ed esegue `test_binaryheap`;
- `make clean`: rimuove artefatti di compilazione ed eseguibili.

9.3 Scelta delle flag di compilazione

Le flag configurate sono:

```
-std=c11 -Wall -Wextra -pedantic
```

Su richiesta esplicita, non sono state abilitate ottimizzazioni del compilatore. Questa scelta è utile in una fase didattica o di sviluppo iniziale per diversi motivi:

- mantiene il comportamento più prevedibile;
- facilita il debugging;
- evita trasformazioni aggressive che potrebbero complicare l'analisi del codice generato.

10 Struttura Finale dei File

Al termine del lavoro, i file principali del progetto risultano essere:

- `binaryheap.c`: implementazione della struttura dati;
- `binaryheap.h`: interfaccia pubblica;
- `binaryheap_demo.c`: programma dimostrativo;
- `test_binaryheap.c`: file di test automatici;
- `Makefile`: build automation;
- `report/report.tex`: questa relazione.

11 Valutazione del Lavoro Svolto

11.1 Risultati raggiunti

Il lavoro ha prodotto una struttura dati funzionante, generica e testata, costruita a partire da un'analisi concreta dei sorgenti `glibc`. Gli obiettivi principali sono stati raggiunti:

- identificazione dell'assenza di una priority queue generica in `glibc`;
- individuazione di un riferimento algoritmico utile;
- implementazione di una min-heap generica in C;
- predisposizione di un ambiente minimo di build e test.

11.2 Limiti attuali

La soluzione attuale è adeguata per lo scopo richiesto, ma presenta alcuni limiti naturali:

- non gestisce direttamente la deallocazione degli oggetti memorizzati;
- non include operazioni come `decrease-key`, `remove` arbitrario o aggiornamento di priorità;
- non è presente una suite di test ancora più estesa con scenari stressati o casuali;
- non è stata prodotta una libreria statica o condivisa separata.

11.3 Possibili estensioni

Tra le evoluzioni più naturali del progetto si possono indicare:

- aggiunta di test con sequenze miste di `push` e `pop`;
- supporto a max-heap tramite wrapper o comparatori dedicati;
- introduzione di un indice esterno per supportare `decrease-key`;
- produzione di una libreria `.a`;
- documentazione aggiuntiva dell'API in formato header commentato o manuale.

12 Conclusione

L'attività svolta ha seguito un percorso tecnico coerente:

1. verifica della disponibilità dei sorgenti `glibc`;
2. analisi per cercare strutture dati equivalenti a una priority queue o binary heap;
3. identificazione di `frame_heapsort` come miglior riferimento algoritmico;
4. progettazione di una heap generica basata su `void **` e comparatore;

5. realizzazione dell'implementazione e della relativa interfaccia;
6. aggiunta di demo, test e automazione della build;
7. stesura della presente relazione.

Il risultato è un piccolo progetto C pulito e coerente, nato da un'esigenza pratica e sviluppato con un approccio incrementale: prima l'analisi, poi la progettazione, infine implementazione e verifica.